

Микросервисы — Шпаргалка

1. Монолит vs Микросервисы

	Монолит	Микросервисы
Структура	1 процесс, 1 база, 1 деплой	N сервисов, N баз, N деплоев
Вызовы	В памяти (наносекунды)	По сети (миллисекунды)
Транзакции	ACID из коробки	Нет общих транзакций → Saga
Масштабирование	Только целиком	По частям
Отказ	1 сбой = падает всё	Изолированные отказы
Стек	Один на весь проект	Любой под задачу
Старт	Просто	Сложно
Рост	Сложно при росте	Гибко при росте

Ключевой принцип: Вертикальная нарезка (по доменам: Orders, Payments, Users) — путь к микросервисам. Горизонтальная (по слоям: Controllers, Services, Repos) — тупик, потому что любое изменение пронзает все слои насквозь.

2. Модульный монолит

Что это: Монолит с вертикальной нарезкой и строгими границами между модулями. 1 процесс, но модули изолированы как отдельные сервисы.

3 правила:

- Модули общаются только через интерфейсы — Order Module вызывает `IPaymentService`, не лезет в классы Payment Module
- У каждого модуля своя схема БД (логически) — таблицы `orders` принадлежат Order Module, `payments` — Payment Module
- Модуль не знает внутренностей другого — только публичный API: `ProcessPayment(orderId, amount)`

Когда использовать:

- Команда 10-50 разработчиков
- Монолит создаёт боль (конфликты в коде, долгая регрессия)
- Микросервисы преждевременны (нет экспертизы/инфраструктуры/времени)
- Хотите подготовиться к будущей миграции

Путь миграции в микросервисы:

- Определить самый нагруженный/изолированный модуль
- Заменить интерфейс на HTTP/gRPC клиент
- Вынести в отдельный репозиторий и развернуть как сервис
- Мигрировать данные в отдельную БД
- Тестировать → повторить для следующего модуля

3. Нарезка сервисов (DDD)

Главная идея: Разбиение по бизнес-функциям, не по техническим слоям.

Эвристики разбиения:

Эвристика	Смысл
Business Capability	Бизнес-функция → отдельный сервис
Data Ownership	Общие данные → один сервис
Change Frequency	Часто меняется → выделить
Scalability Needs	Разная нагрузка → разнести
Team Structure	1 сервис = 1 команда

Когда разделять: разные домены, разные команды, разная нагрузка, меняются независимо.

Когда НЕ разделять: общие данные, всегда меняются вместе, разделение создаст слишком много сетевых вызовов.

4. Database per Service

Железное правило: Каждый сервис — своя БД. Order Service НЕ делает SELECT из базы Payment Service. Никогда.

Почему: Без этого теряется автономность — главное преимущество микросервисов. Payment-команда не сможет менять схему, менять СУБД, масштабировать базу независимо.

Как общаться: Только через API → GET /payments/status?orderId=123

Цена изоляции:

- Нет JOIN через базы → два запроса + склейка на стороне приложения
- Нет транзакций через базы → Saga Pattern
- Eventual Consistency → данные временно расходятся между сервисами

5. CAP-теорема

Суть: Из трёх свойств распределённой системы можно выбрать только два.

Свойство	Расшифровка
C — Consistency	Все узлы видят одинаковые данные одновременно
A — Availability	Система всегда отвечает на запросы
P — Partition Tolerance	Система работает при разрыве сети

В микросервисах P неизбежна (сеть ненадёжна) → выбор между C и A:

	CP (согласованность)	AP (доступность)
Примеры БД	MongoDB (majority), HBase, Consul	Cassandra, DynamoDB, Riak
При сбое	Отказ в записи до восстановления связи	Продолжает работу, данные временно расходятся
Когда	Банки, бронирования, инвентарь	Соцсети, рекомендации, каталоги

Вывод: Чаще всего выбирают AP + Eventual Consistency. Для денег и бронирований — CP + Saga Pattern.

6. Saga Pattern

Суть: Цепочка локальных транзакций с компенсациями. Если шаг N упал → откатить шаги N-1, N-2, ...

Пример — оформление заказа:

1. Order Service → создать заказ (Pending)
2. Payment Service → списать деньги
3. Inventory Service → зарезервировать товар
4. Shipping Service → создать доставку

Если на шаге 3 товара нет → Payment возвращает деньги (компенсация) → Order помечается Failed.

Два типа:

	Хореография	Оркестровка
Управление	Нет центра, сервисы реагируют на события	Центральный оркестратор управляет шагами
Плюсы	Нет единой точки отказа, слабая связанность	Весь флоу в одном месте, легко дебажить
Минусы	Сложно понять полный флоу, трудно дебажить	Оркестратор = точка отказа
Когда	Простые флоу (2-3 шага)	Сложные флоу (5+ шагов), условные ветвления
Аналогия	Бригада строителей — каждый знает свой шаг	Свадебный организатор — координирует всех

На практике: Оркестрация для критичных процессов (заказ, оплата), хореография для фоновых (аналитика, рекомендации).

7. Синхронная vs Асинхронная связь

Синхронная (REST / gRPC) — сервис ЖДЁТ ответа.

Когда: нужен немедленный ответ — аутентификация (`POST /login`), получение профиля (`GET /user/{id}`), проверка наличия товара, валидация платежа с банком.

Минусы: каскадные падения, +5-50ms латентности на каждый вызов.

Асинхронная (Kafka / RabbitMQ) — сервис отправляет событие и ПРОДОЛЖАЕТ работу.

Когда: допустима задержка — email-уведомления, обработка видео, обновление рекомендаций, аналитика и логирование.

Плюсы: низкая связанность, отказоустойчивость, события не теряются.

Правило: Пользователь ждёт → sync. Фоновая задача → async.

На практике используют оба: создание заказа — sync, отправка email — async, обновление статистики — async.

8. Надёжность доставки

3 гарантии доставки:

Гарантия	Суть	Когда
At Most Once	Может потеряться, но без дублей	Логи, метрики
At Least Once	Точно дойдёт, но возможны дубли	Большинство бизнес-операций
Exactly Once	Ровно 1 раз	Финансы (сложно и медленно)

На практике: At Least Once + защита от дублей на стороне получателя.

Outbox Pattern — гарантирует доставку события:

- Событие записывается в таблицу `outbox` в той же транзакции, что и бизнес-данные
- Отдельный процесс (CDC) читает `outbox` → публикует в Kafka
- Результат: событие не потеряется

Idempotency Key — защищает от дублей:

- Клиент присылает уникальный ID запроса: `X-Idempotency-Key: abc123`
- Сервер проверяет в таблице `inbox`
- Повторный запрос → возврат сохранённого результата, а не повторное выполнение